

Advanced Physical Computing 2025-2026
Capstone Project

Smart Fridge Inventory & Recipe Assistant

Submitted by

Ethan Liu

Shanghai American School Puxi

Date: April 2026

Platform: Raspberry Pi 5 + Hailo 8L AI HAT+ + Ollama

Table of Contents

Executive Summary	1
1. Problem Statement	2
1.1 The Problem	2
1.2 Target Customer.....	2
1.3 Solution Usage	2
2. Research.....	2
2.1 What already exists	2
2.2 What I Learned from Research.....	3
3. System Architecture	3
3.1 Component List:.....	3
3.2 Data Flow:.....	4
3.3 Block Diagram	5
4. Wiring Diagram & Pin Assignment.....	5
4.1 Raspberry Pi Pin Assignment Table	5
4.2 PCA9685 PWM Driver Connections	6
4.3 HW-131 Power Module.....	6
5. Code Documentation & Explanation	7
5.1 Version History	7
5.2 Final Code (v6)	9
Left panel — camera feed	10
Right panel — info.....	10
6. Fusion 360 Design Files & Iteration Notes.....	15
6.1 Base Plate – 2 iterations.....	15
6.2 Arm Links – 2 iterations	16
7. Testing Logs.....	17
8. AI Usage Log	19

9. Reflection 20

Executive Summary

The Smart Fridge Assistant is an AI-powered physical computing system that automatically identifies food ingredients in a refrigerator and generates cuisine-specific recipe suggestions. The system addresses a common daily problem: people open their fridge not knowing what to cook, leading to food waste, duplicate grocery purchases, and unnecessary takeout.

The system consists of a 3D-printed robotic arm mounted on a laser-cut base plate on top of a refrigerator. When the user presses the deploy button, the arm swings down into the fridge opening using four MG90S servos driven by a PCA9685 PWM driver. A USB webcam at the end of the arm captures the fridge interior. A custom-trained YOLOv8 object detection model running on the Raspberry Pi 5 identifies food items including eggs, potatoes, onions, carrots, garlic, mushrooms, cabbage, chicken, cucumber, and zucchini. The detected ingredients are passed to Ollama running Llama 3.2 3B locally on the Pi, which generates a recipe suggestion tailored to the user's selected cuisine preference (Chinese, Italian, or Japanese). The recipe — including dish name, preparation method, and any missing ingredients — is displayed on screen. Pressing the deploy button again retracts the arm to its parked position.

The system runs entirely offline with no cloud dependency, costs approximately \$150 in hardware compared to \$3,000+ smart fridge solutions, and responds in under 30 seconds from deploy to recipe display.

Key technical challenges overcome included Pi 5 I2C bus incompatibility with the LCD display, servo torque limitations requiring an upgrade from SG90 to MG90S, power instability from inadequate common grounding, and camera initialization timing issues. The Hailo-8 AI HAT+ is integrated into the system and detected successfully, providing the hardware foundation for future YOLOv8 NPU acceleration planned as an extension of this project.

1. Problem Statement

1.1 The Problem

People open their fridge multiple times a day not knowing what they have, what's running low, or what they can cook with what's available. This leads to food waste (buying duplicates of things already in the fridge), unnecessary grocery trips, and the daily frustration of staring into the fridge trying to decide what to make.

1.2 Target Customer

Anyone who cooks at home, particularly students living independently, busy families, and households where multiple people share a fridge and nobody tracks inventory.

1.3 Solution Usage

A camera mounted at the fridge door automatically scans visible food items when the door opens, identifies them using AI vision, and immediately suggests recipes based on what's available. A physical indicator flags when stock is critically low.

2. Research

2.1 What already exists

- Samsung Family Hub fridge — built-in interior camera, app-based viewing, cloud processing, no recipe generation. Costs \$3000+, requires internet, camera is fixed and fogs up from condensation.
- LG ThinQ fridge — similar concept, app-based, cloud dependent, expensive.
- Grocy / pantry apps — manual barcode scanning to log inventory. Relies entirely on user discipline — most people stop using it within a week.
- ChatGPT / recipe apps — require you to manually type what you have. No automatic detection.
- Fixed fridge cameras (e.g. FridgeCam) — small camera that mounts inside and photos contents when door closes. Still suffers from condensation, fixed angle, no AI processing.

2.2 What I Learned from Research

- COCO-trained YOLOv8 can detect ~20 common food items out of the box (banana, apple, orange, sandwich, bottle, cup, bowl, cake, carrot, hot dog, pizza, donut, broccoli) — sufficient for a meaningful demo
- Ollama can run Llama 3.2 or Gemma 3 locally on Pi 5 — response time is ~3-8 seconds for a short recipe suggestion, acceptable for this use case
- PCA9685 can drive multiple servos simultaneously over I2C from just 2 GPIO pins, solving the PWM pin limitation on Pi 5
- A 2-servo arm (base rotation + shoulder extension) is sufficient to move a lightweight USB camera from a parked position on top of the fridge to a scanning position inside the fridge opening

[Example] The following, Figure 1, depicts the three-view of the final unmanned system design.

[ADD FIGURE-CENTERED]

Figure 1. Three-view of final uncrewed system design.

3. System Architecture

3.1 Component List:

Component	Part	Role
Raspberry Pi 5 (16GB)	RPi 5 16GB	Main compute
Hailo 8L AI HAT+	13 TOPS NPU	Runs YOLOv8 food detection
USB Webcam	Logitech C270 or similar	Captures fridge interior
Push button (deploy)	Momentary tactile button	Triggers arm deploy/retract cycle
Push button (Chinese)	Momentary tactile button	Selects Chinese cuisine preference
Push button (Italian)	Momentary tactile button	Selects Italian cuisine preference

Push button (Japanese)	Momentary tactile button	Selects Japanese cuisine preference
PCA9685 PWM driver	Adafruit PCA9685	Controls both servos over I2C
SG90 Servo 1	SG90	Base rotation (arm swings out over fridge)
SG90 Servo 2	SG90	Shoulder (arm extends down into fridge)
LCD 16x2 (I2C)	LCD1602 with I2C backpack	Displays detected items + recipe
3D printed arm	Bambu Lab PLA	Arm links + camera mount at end effector
Laser cut base plate	3mm acrylic	Mounts PCA9685, Pi, and arm base to fridge top

3.2 Data Flow:

Cuisine button pressed (Chinese / Italian / Japanese)



Pi 5 stores selected cuisine preference



Deploy button pressed



GPIO detects press → arm deploys

(PCA9685 → Servo 1 rotates out, Servo 2 extends down)



Webcam captures frame of fridge interior



Hailo 8L runs YOLOv8 inference on frame



Detected food items list (class labels + confidence scores)



Filter detections by confidence threshold (>0.5)



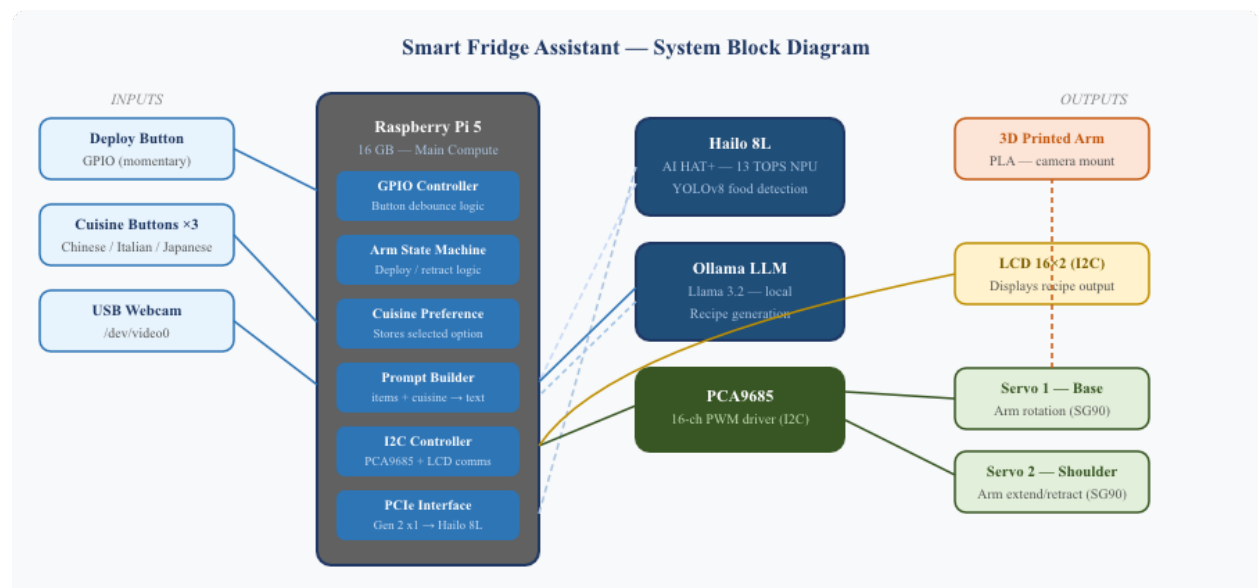
Item list + cuisine preference formatted as prompt

→ sent to Ollama (Llama 3.2):

"Given these ingredients: [items]. Suggest a [cuisine] recipe."

↓
Ollama returns recipe suggestion
↓
LCD displays recipe (scrolling if text exceeds 2 lines)
↓
Deploy button pressed again → arm retracts to parked position
(PCA9685 → Servo 2 retracts, Servo 1 rotates back)

3.3 Block Diagram



4. Wiring Diagram & Pin Assignment

4.1 Raspberry Pi Pin Assignment Table

GPIO (BCM)	Physical Pin	Connected To	Function
GPIO 2 (SDA)	Pin 3	PCA9685 SDA, LCD SDA	I2C data
GPIO 3 (SCL)	Pin 5	PCA9685 SCL, LCD SCL	I2C clock
GPIO 17	Pin 11	Button module OUT 1	Deploy button
GPIO 27	Pin 13	Button module OUT 2	Chinese cuisine
GPIO 22	Pin 15	Button module OUT 3	Italian cuisine

GPIO 23	Pin 16	Button module OUT 4	Japanese cuisine
3.3V	Pin 1	Button module VCC	Button logic power
5V	Pin 2	LCD VCC	LCD panel power
GND	Pin 6	Common ground bus	Shared ground
USB-A	—	USB Webcam	Camera input
PCIe FPC	—	Hailo-8 AI HAT+	NPU connection
HDMI	—	Monitor	Display output

4.2 PCA9685 PWM Driver Connections

PCA9685 Pin	Connected To	Notes
VCC	HW-131 3.3V output	Logic power — NOT Pi 3.3V (insufficient current)
GND	Common ground bus	Must share ground with Pi
SDA	GPIO 2 (Pi pin 3)	I2C data
SCL	GPIO 3 (Pi pin 5)	I2C clock
V+	HW-131 5V output	Servo power rail — external supply required
CH1 PWM	MG90S Servo (middle joint 1)	Signal wire only
CH2 PWM	MG90S Servo (base joint 1)	Signal wire only
CH3 PWM	MG90S Servo (middle joint 2)	Signal wire only
CH4 PWM	MG90S Servo (base joint 2)	Signal wire only

4.3 HW-131 Power Module

HW-131 Pin	Connected To	Notes
Input+	18650 battery pack positive	7.4V input from 2× NCR18650B
Input-	18650 battery pack negative	—
5V output	PCA9685 V+	Servo power
3.3V output	PCA9685 VCC	Logic power
GND	Pi GND (pin 6) + PCA9685 GND	Common ground — critical for I2C stability

5. Code Documentation & Explanation

5.1 Version History

5.1.1 *fridge_assistant_v1-v2 (Early Versions)*

Initial integration attempts combining the Hailo GStreamer pipeline with tkinter display and servo control. These versions used the default COCO YOLOv8 model running on the Hailo-8 NPU via the hailo-apps GStreamer pipeline.

Problems encountered:

- GStreamer pipeline requires signal handlers that only work in the main thread — running it in a background thread caused `ValueError: signal only works in main thread`
- Default COCO model detected general objects (person, donut, banana) but not realistic fridge ingredients
- LCD display integration failed due to Pi 5 I2C bus speed incompatibility

5.1.2 *fridge_assistant_v3-v4 (ONNX Transition)*

Replaced Hailo GStreamer pipeline with ultralytics YOLO running the custom-trained best.onnx model directly. This sacrifices NPU acceleration (model runs on CPU) in exchange for reliability and compatibility with threading.

Key change:

- Detection now uses `YOLO('/home/stussy/Downloads/best.onnx', task='detect')` with `onnxruntime` backend. Runs at approximately 1 frame per second on Pi 5 CPU, acceptable for a fridge scanning use case.
- Added custom food classes from trained dataset: cabbage, carrot, chicken, cucumber, egg, garlic, mushroom, onion, potato, zucchini.

5.1.3 *fridge_assistant_v5 (State Machine)*

Introduced a 3-state state machine to control program flow:

STATE_IDLE → (deploy press) → STATE_DEPLOYED → (deploy press) → STATE_RECIPE → (deploy press) → STATE_IDLE

- STATE_IDLE: Arm parked, waiting for user input
- STATE_DEPLOYED: Arm down, camera scanning, detections accumulate across frames
- STATE_RECIPE: Items locked, Ollama generating recipe, displaying result

Detection accumulation fix: Changed `user_data.live_items = list(set(items))` to `user_data.live_items = list(set(user_data.live_items + items))` so items detected across multiple frames are preserved rather than replaced each frame.

Ollama integration: Uses `subprocess.run(['ollama', 'run', 'llama3.2:3b', prompt])` with 120s timeout. Prompt engineered to return dish name, cooking description, and missing ingredients in 3 sentences.

5.1.4 fridge_assistant_v6 (Final Version)

Final integrated script with all features working:

Architecture — 3 concurrent threads:

- Main thread: tkinter event loop and UI updates
- `run_camera`: captures frames at 30fps, stores in `user_data.latest_frame`
- `run_detection`: runs YOLO inference at 1fps using latest frame, updates detected items

Key Functions:

- `move_joint(indices, targets, steps, delay)` — smooth servo interpolation. Takes position indices (0-3) mapped through `CHANNELS=[4,1,2,3]` to actual PCA9685 channels. Moves in 15 steps with 0.08s delay between steps for smooth motion. Includes retry logic for I2C errors.
- `deploy_arm()` / `retract_arm()` — sequence control. Deploy moves middle joint first then base to avoid mechanical collision. Retract reverses: middle first, then base.
- `on_deploy()` — state machine transition handler. Single button controls all three state transitions depending on current state.

- `get_recipe()` — runs in separate thread to avoid blocking UI. Strips ANSI escape codes from Ollama output using `re.sub(r'\x1b\\[[0-9;]*m', '', recipe)` before displaying.
- `update_camera()` — tkinter `after()` callback running every 50ms, converts OpenCV BGR frame to PIL ImageTk for display in left panel.
- CHANNELS mapping: `CHANNELS = [4, 1, 2, 3]` maps position indices to PCA9685 channels. Index 0 = CH4 (base servo 1), Index 1 = CH1 (middle servo 1), Index 2 = CH2 (base servo 2), Index 3 = CH3 (middle servo 2). CH0 was found to be faulty and replaced with CH4.

5.2 Final Code (v6)

```
import cv2 import subprocess import time import threading import tkinter as tk from PIL import Image,
ImageTk from gpiozero import Button from adafruit_servokit import ServoKit from ultralytics import
YOLO
```

SERVO SETUP

```
kit = ServoKit(channels=16) for ch in [1, 2, 3, 4]: kit.servo[ch].set_pulse_width_range(600, 2400)
time.sleep(0.5)
```

```
CHANNELS = [4, 1, 2, 3] PARKED = [65, 125, 97, 20] DEPLOYED = [115, 75, 47, 70]
```

```
pos = PARKED[:]
```

```
def set_servo(ch, angle): angle = max(0, min(180, int(angle))) for attempt in range(4): try:
kit.servo[ch].angle = angle return except Exception as e: print(f"CH{ch} retry {attempt+1}: {e}")
time.sleep(0.3)
```

```
def move_joint(indices, targets, steps=15, delay=0.08): start = pos[:] for i in range(1, steps + 1): for idx in
indices: new = int(start[idx] + (targets[idx] - start[idx]) * i / steps) new = max(0, min(180, new))
set_servo(CHANNELS[idx], new) time.sleep(0.04) time.sleep(delay) for idx in indices: pos[idx] =
targets[idx]
```

```
def deploy_arm(): display("Deploying arm...") mid_targets = pos[:] mid_targets[1] = DEPLOYED[1]
mid_targets[3] = DEPLOYED[3] move_joint([1, 3], mid_targets) time.sleep(0.5) base_targets = pos[:]
base_targets[0] = DEPLOYED[0] base_targets[2] = DEPLOYED[2] move_joint([0, 2], base_targets)
```

```
def retract_arm(): display("Retracting...") base_targets = pos[:] base_targets[0] = PARKED[0]
base_targets[2] = PARKED[2] move_joint([0, 2], base_targets) time.sleep(0.5) mid_targets = pos[:]
mid_targets[1] = PARKED[1] mid_targets[3] = PARKED[3] move_joint([1, 3], mid_targets)
```

DISPLAY SETUP

```
root = tk.Tk() root.title("Fridge Assistant") root.configure(bg="black") root.attributes('-fullscreen', True)
root.update() SW = root.winfo_width() SH = root.winfo_height()
```

Left panel — camera feed

```
cam_label = tk.Label(root, bg="black") cam_label.place(x=0, y=0, width=SW//2, height=SH)
```

Right panel — info

```
right = tk.Frame(root, bg="black") right.place(x=SW//2, y=0, width=SW//2, height=SH)
```

```
tk.Label(right, text="Fridge Assistant", font=("Arial", 32, "bold"), bg="black", fg="white").pack(pady=20)
```

```
cuisine_label = tk.Label(right, text="Cuisine: Chinese", font=("Arial", 22), bg="black", fg="yellow")
cuisine_label.pack(pady=5)
```

```
status_label = tk.Label(right, text="Select cuisine → Deploy", font=("Arial", 18), bg="black", fg="green")
status_label.pack(pady=5)
```

```
detected_label = tk.Label(right, text="", font=("Arial", 16), bg="black", fg="cyan", wraplength=SW//2 -
40) detected_label.pack(pady=5)
```

```
recipe_label = tk.Label(right, text="", font=("Arial", 15), bg="black", fg="white", wraplength=SW//2 - 40,
justify="left") recipe_label.pack(pady=10, padx=20)
```

```
hint_label = tk.Label(right, text="", font=("Arial", 14, "italic"), bg="black", fg="gray")
hint_label.pack(pady=5)
```

QUIT BUTTON

```
quit_btn = tk.Button(right, text="× Quit", font=("Arial", 14), bg="#333333", fg="white", relief="flat",
command=lambda: quit_program()) quit_btn.pack(side="bottom", pady=20)
```

```
def display(status="", recipe="", detected="", hint=""): status_label.config(text=status)
recipe_label.config(text=recipe) detected_label.config(text=detected) hint_label.config(text=hint)
root.update()
```

```
def quit_program(): print("Quitting...") retract_arm() root.destroy()
```

STATE

```
STATE_IDLE = 0 STATE_DEPLOYED = 1 STATE_RECIPE = 2 state = STATE_IDLE
```

```
class UserData: def init(self): self.live_items = [] self.locked_items = [] self.cuisine = "Chinese"
self.latest_frame = None
```

```
user_data = UserData()
```

CAMERA FEED UPDATE

```
def update_camera(): if user_data.latest_frame is not None: frame = user_data.latest_frame.copy()
frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) h, w = frame.shape[:2] # Scale to fit left panel
panel_w = SW // 2 panel_h = SH scale = min(panel_w / w, panel_h / h) new_w = int(w * scale) new_h =
int(h * scale) frame = cv2.resize(frame, (new_w, new_h)) img =
ImageTk.PhotoImage(Image.fromarray(frame)) cam_label.config(image=img) cam_label.image = img
root.after(50, update_camera)
```

DETECTION LOOP

```
def run_detection(): model = YOLO('/home/stussy/Downloads/best.onnx', task='detect') cap =
cv2.VideoCapture(0) FOOD_CLASSES = { "cabbage", "carrot", "chicken", "cucumber", "egg", "garlic",
"mushroom", "onion", "potato", "zucchini" }
```

```

while True:
    ret, frame = cap.read() if not ret:
        time.sleep(0.1)
        continue

    if state == STATE_DEPLOYED:
        results = model(frame, verbose=False)
        items = []
        for r in results:
            for box in r.bboxes:
                label = model.names[int(box.cls)]
                conf = float(box.conf)
                if conf > 0.35 and label in FOOD_CLASSES:
                    items.append(label)
                    # Draw box on frame
                    x1, y1, x2, y2 = map(int, box.xyxy[0])
                    cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 255), 2)
                    cv2.putText(frame, f"{label} {conf:.2f}",
                                (x1, y1 - 8), cv2.FONT_HERSHEY_SIMPLEX,
                                0.5, (0, 255, 255), 2)

        if items:
            user_data.live_items = list(set(user_data.live_items + items))
            items_str = ", ".join(sorted(user_data.live_items))
            detected_label.config(text=f"Detected: {items_str}")
            root.update()

    user_data.latest_frame = frame
    time.sleep(0.05)

```

OLLAMA

```

def get_recipe(): items_str = ", ".join(sorted(user_data.locked_items)) display( "Generating recipe...",
detected=f"Locked: {items_str}", hint="Please wait..." ) prompt = ( f"I found these ingredients in my
fridge: {items_str}. " f"Suggest ONE simple {user_data.cuisine} recipe I could make. " f"In 3 sentences:
name the dish, briefly describe how to make it, " f"then list any extra ingredients I would need." ) try:
result = subprocess.run( ["ollama", "run", "llama3.2:3b", prompt], capture_output=True, text=True,
timeout=120 ) recipe = result.stdout.strip() display( f"{user_data.cuisine} Recipe", recipe=recipe,
detected=f"Ingredients: {items_str}", hint="Press Deploy to retract arm" ) except Exception as e:
display("Ollama error", recipe=str(e)[:80])

```

BUTTON CALLBACKS

```

def on_deploy(): global state
if state == STATE_IDLE:
    state = STATE_DEPLOYED
    user_data.live_items = []
    def do_deploy():
        deploy_arm()
        display(
            "Scanning fridge...",
            detected="Looking for food...",
            hint="Press Deploy again to lock items & get recipe"
        )
    threading.Thread(target=do_deploy, daemon=True).start()

elif state == STATE_DEPLOYED:
    if not user_data.live_items:
        display("No food detected!", hint="Try adjusting camera angle")
        return
    state = STATE_RECIPE
    user_data.locked_items = user_data.live_items[:]
    threading.Thread(target=get_recipe, daemon=True).start()

elif state == STATE_RECIPE:

```



```

state = STATE_IDLE
user_data.live_items = []
user_data.locked_items = []
def do_retract():
    retract_arm()
    display("Ready!", hint="Select cuisine then press Deploy")
threading.Thread(target=do_retract, daemon=True).start()

```

```

def on_chinese(): user_data.cuisine = "Chinese" cuisine_label.config(text="Cuisine: Chinese")
root.update()

```

```

def on_italian(): user_data.cuisine = "Italian" cuisine_label.config(text="Cuisine: Italian") root.update()

```

```

def on_japanese(): user_data.cuisine = "Japanese" cuisine_label.config(text="Cuisine: Japanese")
root.update()

```

```

btn_deploy = Button(17, pull_up=False) btn_chinese = Button(27, pull_up=False) btn_italian =
Button(22, pull_up=False) btn_japanese = Button(23, pull_up=False)

```

```

btn_deploy.when_pressed = on_deploy btn_chinese.when_pressed = on_chinese
btn_italian.when_pressed = on_italian btn_japanese.when_pressed = on_japanese

```

STARTUP

```

print("Starting Fridge Assistant...")

```

```

for i, ch in enumerate(CHANNELS): set_servo(ch, PARKED[i]) time.sleep(1.5)

```

```

display("Fridge Assistant ready!", hint="Select cuisine then press Deploy")

```

```

detection_thread = threading.Thread(target=run_detection, daemon=True) detection_thread.start()

```

```

root.after(100, update_camera)

```

MAIN LOOP

```
try: root.mainloop() except KeyboardInterrupt: print("\nShutting down...") retract_arm() root.destroy()
```

6. Fusion 360 Design Files & Iteration Notes

6.1 Base Plate – 2 iterations

6.1.1 Iteration 1 (Cardboard)

Final integrated script with all features working:

A rough cardboard prototype with approximate hole positions for the base servos and PCA9685. This was used to verify the general layout and spacing before committing to a permanent material. Holes were laser-cut and approximate.

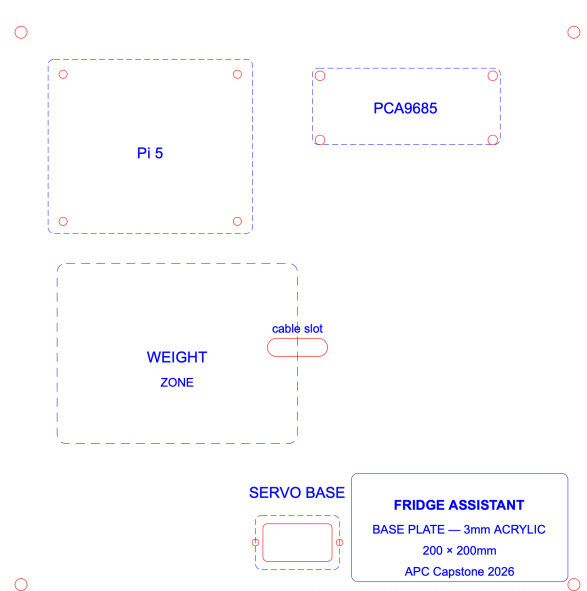


Figure 2. Iteration 1 of the Base Plate

6.1.2 Iteration 2 (Wood)

Final integrated script with all features working:

A rough cardboard prototype with approximate hole positions for the base servos and PCA9685. This was used to verify the general layout and spacing before committing to a permanent material. Holes were hand-cut and approximate — no precise measurements.

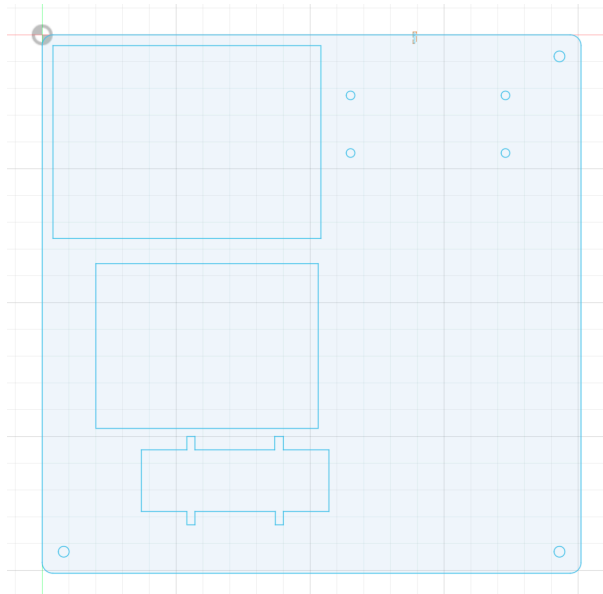


Figure 2. Iteration 2 of the Base Plate

Key learning: The cardboard prototype saved significant time — discovering that the servo spacing needed adjustment before cutting the final material avoided wasting a wood sheet.

6.2 Arm Links – 2 iterations

6.2.1 Iteration 1

Simple rectangular beam with flat ends. Connected to servo horns using hot glue — unreliable under torque and came loose during testing.

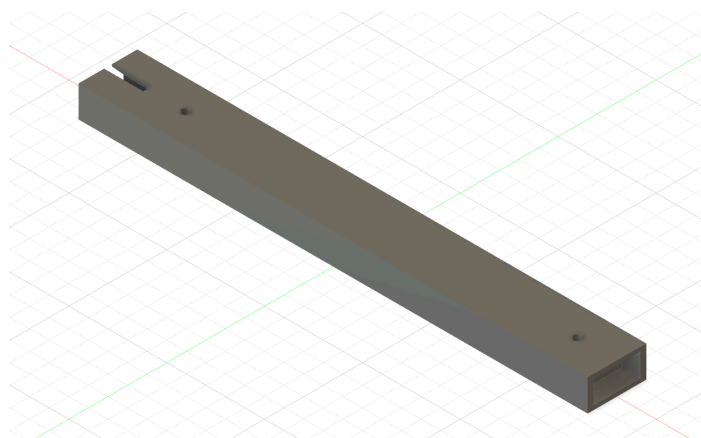


Figure 1. Iteration 1 of the Arm Link

6.2.2 Iteration 2

Added recessed slots at the joint ends sized to the servo horn profile, so the horn sits flush and is mechanically located before any adhesive is applied. For the dual-servo base joint, replaced the simple flat plate with measured slots that the MG90S servo bodies press-fit into precisely, eliminating reliance on hot glue for alignment.

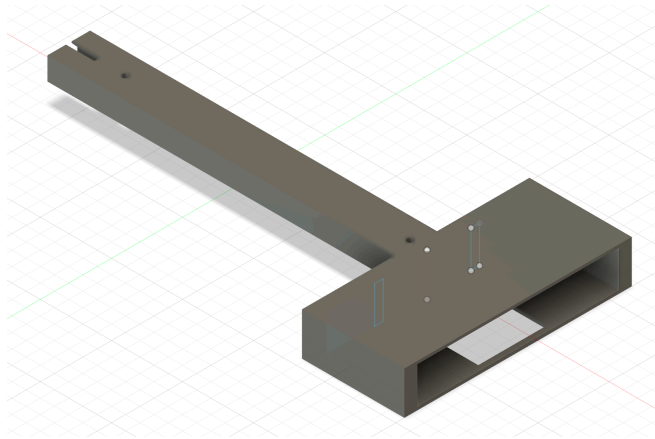


Figure 2. Iteration 2 of the Base Plate

Key learning: Mechanical location is far more reliable than adhesive alone under repeated servo movement. The press-fit slots made the joint significantly more stable.

7. Testing Logs

Date	Component	What was tested	Result	Fix applied
Mar	LCD (16×2 I2C)	Display text via RPLCD	Failed — OSError Errno 121 Remote I/O error	Discovered Pi 5 bcmstb I2C controller runs at 97.5kHz — too fast for PCF8574T. Attempted baudrate config, software I2C overlay.

				Unresolved — pivoted to HDMI display
Mar	PCA9685	i2cdetect	Detected at 0x40 ✓	—
Mar	SG90 servos × 4	Basic angle movement	Moved correctly individually	—
Mar	SG90 servos × 4	All 4 moving simultaneously	I2C crashed mid-run (Errno 121)	Fixed common ground between HW-131 power module and Pi — voltage difference was causing bus instability
Mar	SG90 servos	Arm lift test	Servos overheated and stalled — insufficient torque to lift arm weight	Upgraded to MG90S metal gear servos
Mar	MG90S servos	Arm lift test	Successfully lifted arm ✓	Calibrated PARKED and DEPLOYED angles per channel
Apr	Hailo-8	hailortcli fw-control identify	Detected: Firmware 4.23.0, HAILO8 architecture ✓	—
Apr	Buttons × 4	GPIO input test	Initially read as always pressed	Changed pull_up=True to pull_up=False — button modules are active HIGH
Apr	Ollama	Recipe generation	Worked but slow (~30s) and initial timeout errors	Increased timeout to 120s, shortened prompt
Apr	Ollama output	Display on screen	Random characters appeared (□[7D□[K)	Stripped ANSI escape codes with regex
May	USB webcam	cv2.VideoCapture(0)	Failed — camera not found	Added 2-second delay after opening — Pi needs time to initialize USB camera

May	Custom YOLOv8 (5 epochs)	Potato detection	Detected as onion/donut	Retrained for 10 epochs — improved but still imperfect
May	Camera feed	Display in tkinter window	Frame frozen — detection blocking UI thread	Separated into two threads: run_camera (30fps) and run_detection (1fps)
May	Full system	End-to-end flow	Working ✓	—

8. AI Usage Log

This project was developed with significant AI assistance through Claude (Anthropic). Below is a summary of how AI was used and the decisions made:

- Hardware debugging
 - AI was used extensively to diagnose the I2C LCD issue. Claude identified that the Pi 5's new brcmstb I2C controller runs at a fixed 97.5kHz and doesn't respond to standard baudrate configuration — a known Pi 5-specific issue. Multiple approaches were suggested (baudrate config, software I2C overlay, different GPIO pins). When all failed, the decision to pivot to HDMI display was made by the student after weighing time cost vs. benefit.
- Code architecture
 - Claude generated the initial script structure including the state machine (IDLE → DEPLOYED → RECIPE), servo movement functions with smooth interpolation, and the threading architecture separating camera capture from detection. The student made decisions about servo channel assignments, calibrated angles, and button GPIO pins based on physical testing.
- Servo calibration
 - The calibration process was entirely student-driven — the student ran test scripts, physically observed servo movement, and reported back observed angles. AI provided the test code framework but the student determined all PARKED and DEPLOYED values through hands-on iteration.
- Detection and Ollama
 - Claude suggested switching from the Hailo GStreamer pipeline to ultralytics ONNX inference when the pipeline couldn't run in a background thread. The

student decided to use a Roboflow public dataset for training rather than building a custom dataset from scratch, based on time constraints. Prompt engineering for Ollama (asking for dish name, description, and missing ingredients) was a collaborative decision.

- Key principle
 - AI was used as a technical reference and code generation tool. All hardware decisions, physical calibration, and design tradeoffs were made by the student based on real-world testing results.

9. Reflection

This project pushed me significantly beyond the scope of standard coursework. Building a system that combines physical robotics (a 4-servo arm driven by a PCA9685), computer vision (custom-trained YOLOv8), and a local language model (Ollama Llama 3.2) into a single integrated pipeline required coordinating hardware, firmware, and software layers simultaneously — a level of complexity I hadn't encountered before.

The most valuable learning was about the gap between theory and practice. The Raspberry Pi 5 introduced unexpected incompatibilities — its new I2C controller, PCIe interface, and GPIO behavior differ from older Pi models in ways that aren't always documented. What should have been a simple LCD connection turned into a multi-session debugging process that ultimately couldn't be resolved within the project timeline. This taught me to research hardware-specific issues before assuming standard approaches will work, and to have contingency plans.

The servo system also taught me about mechanical engineering tradeoffs. SG90 servos were sufficient in isolation but completely inadequate under the load of a 26cm arm with a camera at the end. Upgrading to MG90S and redesigning the 3D printed joints to use mechanical location instead of adhesive was a practical lesson in how real systems fail under conditions that weren't obvious during initial design.

The AI-assisted development process was a genuine productivity multiplier but required careful judgment. Claude could generate working code quickly, but it couldn't know the physical state of my hardware, the exact angles my servos were at, or why a particular wire connection was causing instability. Every AI suggestion had to be verified through physical testing, and many

required modification based on what actually happened. The skill of knowing when to trust AI output and when to question it was as important as any technical knowledge.

If I were to redo this project, I would start with a power budget analysis before ordering any components — the current draw from four servos was a problem I didn't anticipate. I would also prototype the arm mechanism in cardboard before 3D printing to verify the mechanical design, which I did for the base plate but not the arm links, costing me two print iterations.

Overall this project successfully demonstrated that edge AI — running both vision inference and language model generation entirely on local hardware without internet — is genuinely practical at the hobbyist level. The Hailo-8 HAT+ and Raspberry Pi 5 together make what would have required cloud services just a few years ago possible on a \$200 board.